Deliverable 1

Thea Ariansen Gathe, Kiki Julia Titia Beumer

Import libraries

In [41]:
```python
import requests
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import numpy as np
import seaborn as sns
import yaml
```

Task 1: Temperature

Retrieve the user ID and secret of frost.met.no
The format of the txt file should look as follows:
user: <user_ID>
secret: <secret_code>

In [6]:
```python
# Get the user ID and secret from txt file
with open("../../met.txt") as f:
    user_id = f.readline().split(":", 1)[1].strip()
    secret = f.readline().split(":", 1)[1].strip()
```

Define the parameters, request, and store in json file

In [7]:
```python
# Define endpoint and parameters
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
    'sources': 'SN17850',
    'elements': 'mean(air_temperature P1D)',
    'referencetime': '2025-01-01/2025-12-31',
}

# Issue an HTTP GET request
r = requests.get(endpoint, parameters, auth=(user_id, secret))

# Extract JSON data
json_df = r.json()
```

Check if the request worked, print out any errors

In [ ]:
```python
# If returned code = 200, the request was successfull
if r.status_code == 200:
    data = json_df['data']
    print('Data retrieved from frost.met.no!')

# If returned code is not 200, an error occurred during request
else:
    print('Error! Returned status code %s' % r.status_code)
    print('Message: %s' % json_df['error']['message'])
    print('Reason: %s' % json_df['error']['reason'])
```

```
Data retrieved from frost.met.no!
```

Create the dataframe

In [ ]:
```python
# Initialize the dataframe and the rows of the dataframe
df = pd.DataFrame()
rows = []

# For each index add observation data, time and sourceId
for i in range(len(data)):
    row = pd.DataFrame(data[i]['observations'])
    row['referenceTime'] = data[i]['referenceTime']
    row['sourceId'] = data[i]['sourceId']
    rows.append(row)

# Concat all rows to create the dataframe
df = pd.concat(rows, ignore_index=True)

df = df.reset_index()
df.head()
```

Out[ ]:

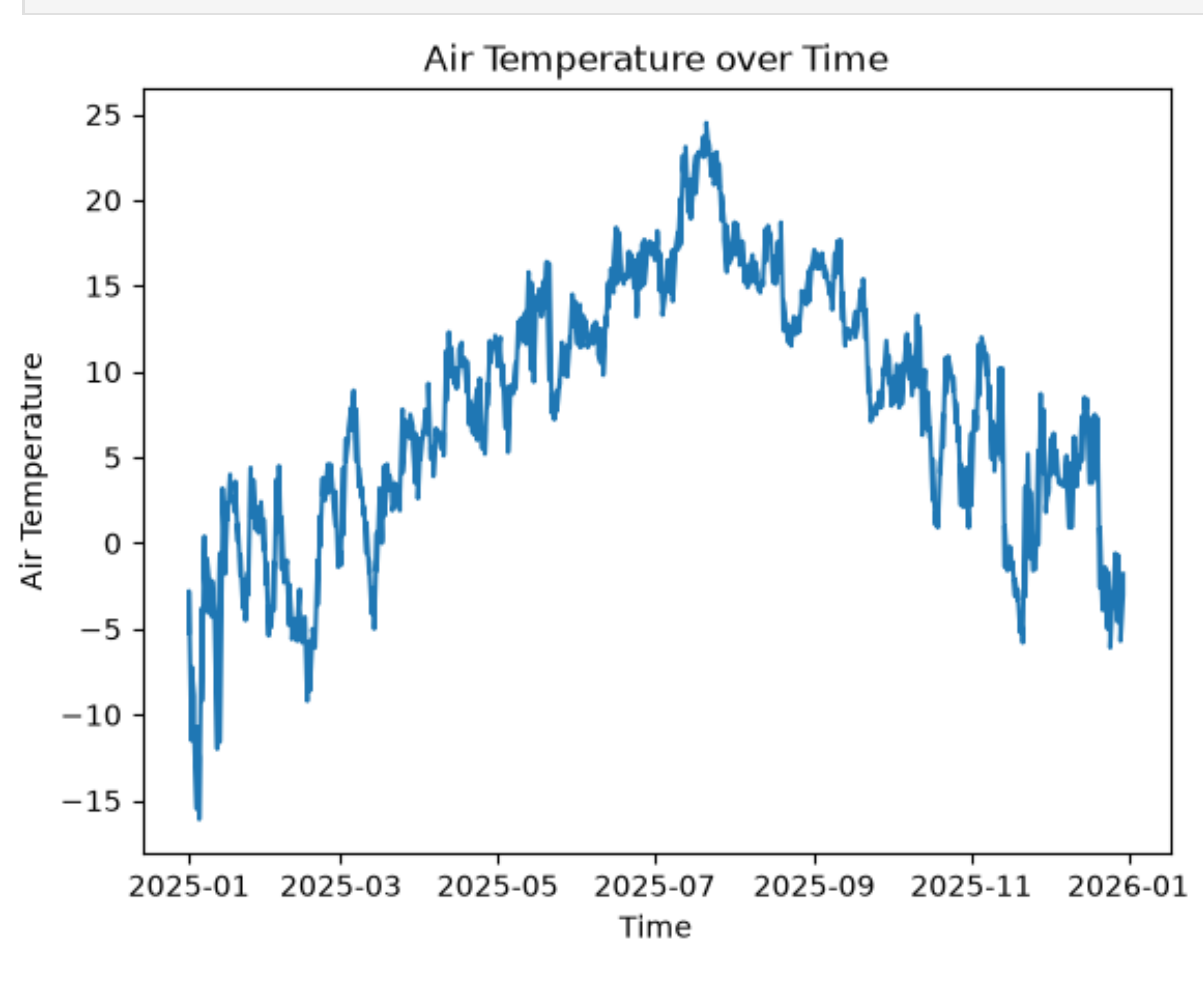
	index	elementId	value	unit	level	timeOffset	timeResolution	timeSeriesId	performanceCategory	exposureCategory	qualityCode	referenceTime	sourceId
0	0	mean(air_temperature P1D)	-5.2	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	0.0	2025-01-01T00:00:00.000Z	SN17850:0
1	1	mean(air_temperature P1D)	-2.8	degC	{'levelType': 'height_above_ground', 'unit': '...	PT6H	P1D	0	C	1	NaN	2025-01-01T00:00:00.000Z	SN17850:0
2	2	mean(air_temperature P1D)	-11.5	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	0.0	2025-01-02T00:00:00.000Z	SN17850:0
3	3	mean(air_temperature P1D)	-7.2	degC	{'levelType': 'height_above_ground', 'unit': '...	PT6H	P1D	0	C	1	NaN	2025-01-02T00:00:00.000Z	SN17850:0
4	4	mean(air_temperature P1D)	-8.9	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	0.0	2025-01-03T00:00:00.000Z	SN17850:0

Make a nicely formatted plot of this air temperature, where time is on the x-axis and temperature is on the y-axis

In [10]:
```python
# Ensure that time is in proper format
df['referenceTime'] = pd.to_datetime(df['referenceTime'])

df.dtypes
```

Out[10]:
```
index                                int64
elementId                           object
value                              float64
unit                                object
level                               object
timeOffset                          object
timeResolution                      object
timeSeriesId                         int64
performanceCategory                 object
exposureCategory                    object
qualityCode                        float64
referenceTime          datetime64[ns, UTC]
sourceId                            object
dtype: object
```

In [ ]:
```python
# Define variables used for x- and y-axis
x = df['referenceTime']
y = df['value']

# Create the plot
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("Air Temperature over Time")
ax.set_xlabel("Time")
ax.set_ylabel("Air Temperature")

plt.show()
```



This plot shows that the air temperature was highest in the summer (july, august). The end of the year 2025, had relatively high temperatures.

Create a nicely formatted summary table containing mean, median, min and max temperature for this year

In [ ]:
```python
# Use describe to obtain the statistics
df['value'].describe()
```

Out[ ]:
```
count    728.000000
mean       7.903571
std        7.528326
min      -16.100000
25%        2.800000
50%        8.350000
75%       13.800000
max       24.500000
Name: value, dtype: float64
```

The average temperature of 2025 was 7.39 degrees. The lowest temperature in 2025 was -16.1 degrees, and the highest air temperature was 24.5 degrees. The distribution seems somewhat left skewed.

Task 2: Precipitation

Request data for precipitation for the same time period, specifically measurements of precipitation over 24 hours.

In [13]:
```python
# Define endpoint and parameters
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
    'sources': 'SN17850',
    'elements': 'sum(precipitation_amount P1D)',
    'referencetime': '2025-01-01/2025-12-31',
}

# Issue an HTTP GET request
r2 = requests.get(endpoint, parameters, auth=(user_id, secret))

# Extract JSON data
json_df2 = r2.json()
```

Check if the request worked, print out any errors

In [ ]:
```python
# If returned code = 200, the request was successfull
if r2.status_code == 200:
    data2 = json_df2['data']
    print('Data retrieved from frost.met.no!')

# If returned code is not 200, an error occurred during request
else:
    print('Error! Returned status code %s' % r2.status_code)
    print('Message: %s' % json_df2['error']['message'])
    print('Reason: %s' % json_df2['error']['reason'])
```

```
Data retrieved from frost.met.no!
```

Create the dataframe

In [ ]:
```python
# Initialize the dataframe and the rows of the dataframe
df2 = pd.DataFrame()
rows2 = []

# For each index add observation data, time and sourceId
for i in range(len(data2)):
    row2 = pd.DataFrame(data2[i]['observations'])
    row2['referenceTime'] = data2[i]['referenceTime']
    row2['sourceId'] = data2[i]['sourceId']
    rows2.append(row2)

# Concat all rows to create the dataframe
df2 = pd.concat(rows2, ignore_index=True)

df2 = df2.reset_index()
df2.head()
```

Out[ ]:

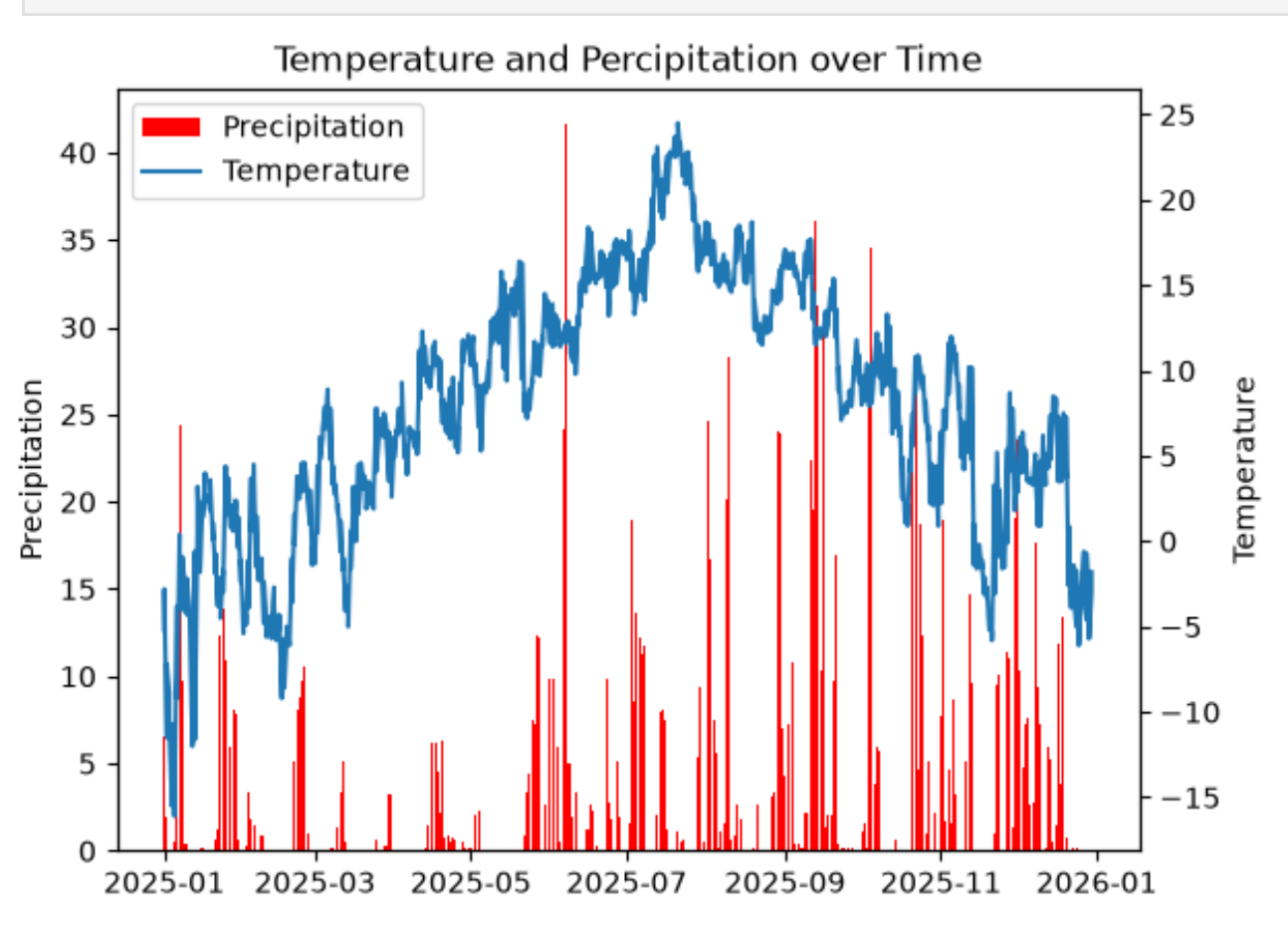
	index	elementId	value	unit	timeOffset	timeResolution	timeSeriesId	performanceCategory	exposureCategory	qualityCode	referenceTime	sourceId
0	0	sum(precipitation_amount P1D)	5.4	mm	PT18H	P1D	0	C	2	NaN	2025-01-01T00:00:00.000Z	SN17850:0
1	1	sum(precipitation_amount P1D)	6.6	mm	PT6H	P1D	0	C	2	0.0	2025-01-01T00:00:00.000Z	SN17850:0
2	2	sum(precipitation_amount P1D)	0.0	mm	PT18H	P1D	0	C	2	NaN	2025-01-02T00:00:00.000Z	SN17850:0
3	3	sum(precipitation_amount P1D)	1.9	mm	PT6H	P1D	0	C	2	0.0	2025-01-02T00:00:00.000Z	SN17850:0
4	4	sum(precipitation_amount P1D)	0.1	mm	PT18H	P1D	0	C	2	NaN	2025-01-03T00:00:00.000Z	SN17850:0

In [16]:
```python
# Ensure that time is in proper format
df2['referenceTime'] = pd.to_datetime(df2['referenceTime'])

df2.dtypes
```

Out[16]:
```
index                                int64
elementId                           object
value                              float64
unit                                object
timeOffset                          object
timeResolution                      object
timeSeriesId                         int64
performanceCategory                 object
exposureCategory                    object
qualityCode                        float64
referenceTime          datetime64[ns, UTC]
sourceId                            object
dtype: object
```

Temperature is continuous, so a line would be appropriate. Precipitation is accumulated over a day, so bars would be best.

In [17]:
```python
fig, ax1 = plt.subplots()

# Bar plot
ax1.bar(df2['referenceTime'], df2['value'], color = 'red', label='Precipitation')
ax1.set_ylabel('Precipitation')

# Line plot
ax2 = ax1.twinx()
ax2.plot(df['referenceTime'], df['value'], label='Temperature')
ax2.set_ylabel('Temperature')

# Combined legend
h1, l1 = ax1.get_legend_handles_labels()
h2, l2 = ax2.get_legend_handles_labels()
ax1.legend(h1 + h2, l1 + l2, loc='upper left')

plt.title('Temperature and Percipitation over Time')
plt.show()
```



The plot shows that there was little precipitation in the spring of 2025. In the autumn, there was a lot of precipitation. In june, there was a short period where there was an extreme amount of precipitation.

Task 3: Historical temperature data

Get the temperatures from 100 years ago

In [18]:
```python
# Define endpoint and parameters
endpoint = 'https://frost.met.no/observations/v0.jsonld'
parameters = {
    'sources': 'SN17850',
    'elements': 'mean(air_temperature P1D)',
    'referencetime': '1925-01-01/1925-12-31',
}

# Issue an HTTP GET request
r3 = requests.get(endpoint, parameters, auth=(user_id, secret))

# Extract JSON data
json_df3 = r3.json()

# Check if request worked
if r3.status_code == 200:
    data3 = json_df3['data']
    print('Data retrieved from frost.met.no!')
else:
    print('Error! Returned status code %s' % r3.status_code)
    print('Message: %s' % json_df3['error']['message'])
    print('Reason: %s' % json_df3['error']['reason'])
```

```
Data retrieved from frost.met.no!
```

Create the dataframe

In [ ]:
```python
# Initialize the dataframe and the rows of the dataframe
df3 = pd.DataFrame()
rows3 = []

# For each index add observation data, time and sourceId
for i in range(len(data3)):
    row3 = pd.DataFrame(data3[i]['observations'])
    row3['referenceTime'] = data3[i]['referenceTime']
    row3['sourceId'] = data3[i]['sourceId']
    rows3.append(row3)

# Concat all rows to create the dataframe
df3 = pd.concat(rows3, ignore_index=True)

df3 = df3.reset_index()
df3.head()
```

Out[ ]:

	index	elementId	value	unit	level	timeOffset	timeResolution	timeSeriesId	performanceCategory	exposureCategory	qualityCode	referenceTime	sourceId
0	0	mean(air_temperature P1D)	1.3	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	2	1925-01-01T00:00:00.000Z	SN17850:0
1	1	mean(air_temperature P1D)	4.1	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	2	1925-01-02T00:00:00.000Z	SN17850:0
2	2	mean(air_temperature P1D)	2.1	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	2	1925-01-03T00:00:00.000Z	SN17850:0
3	3	mean(air_temperature P1D)	0.1	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	2	1925-01-04T00:00:00.000Z	SN17850:0
4	4	mean(air_temperature P1D)	-2.2	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	2	1925-01-05T00:00:00.000Z	SN17850:0

In [20]:
```python
# Ensure that time is in proper format
df3['referenceTime'] = pd.to_datetime(df3['referenceTime'])

df3.dtypes
```

Out[20]:
```
index                                int64
elementId                           object
value                              float64
unit                                object
level                               object
timeOffset                          object
timeResolution                      object
timeSeriesId                         int64
performanceCategory                 object
exposureCategory                    object
qualityCode                          int64
referenceTime          datetime64[ns, UTC]
sourceId                            object
dtype: object
```

Add day and month as column in df and df3

In [21]:
```python
# Remove the year from the datetime
df['month_day'] = df['referenceTime'].dt.strftime('%m-%d')
df3['month_day'] = df3['referenceTime'].dt.strftime('%m-%d')

df3.head(5)
```

Out[21]:

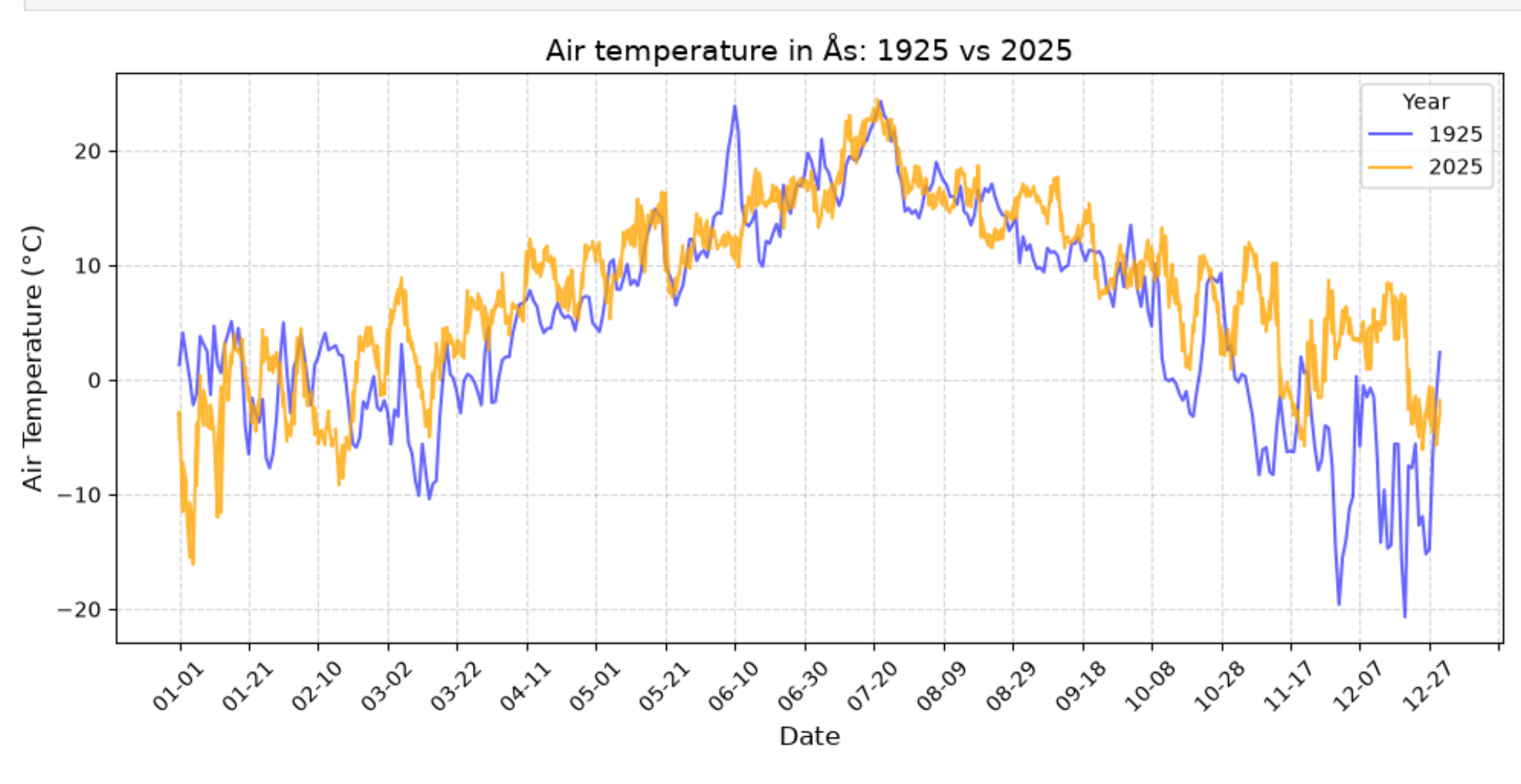
	index	elementId	value	unit	level	timeOffset	timeResolution	timeSeriesId	performanceCategory	exposureCategory	qualityCode	referenceTime	sourceId
0	0	mean(air_temperature P1D)	1.3	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	2	1925-01-01 00:00:00+00:00	SN17850:0
1	1	mean(air_temperature P1D)	4.1	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	2	1925-01-02 00:00:00+00:00	SN17850:0
2	2	mean(air_temperature P1D)	2.1	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	2	1925-01-03 00:00:00+00:00	SN17850:0
3	3	mean(air_temperature P1D)	0.1	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	2	1925-01-04 00:00:00+00:00	SN17850:0
4	4	mean(air_temperature P1D)	-2.2	degC	{'levelType': 'height_above_ground', 'unit': '...	PT0H	P1D	0	C	1	2	1925-01-05 00:00:00+00:00	SN17850:0

In [ ]:
```python
# Check if referenceTime is of the right type
df3.dtypes
```

Out[ ]:
```
index                                int64
elementId                           object
value                              float64
unit                                object
level                               object
timeOffset                          object
timeResolution                      object
timeSeriesId                         int64
performanceCategory                 object
exposureCategory                    object
qualityCode                          int64
referenceTime          datetime64[ns, UTC]
sourceId                            object
month_day                           object
dtype: object
```

Plot the new and the old temperatures in the same plot

In [36]:
```python
# Plot the new and the old temperatures in the same plot
fig, ax = plt.subplots(figsize=(10, 5))

# Plot for 1925
ax.plot(df3['month_day'], df3['value'], label='1925', color='blue', alpha=0.6)

# Plot for 2025
ax.plot(df['month_day'], df['value'], label='2025', color='orange', alpha=0.8)

# Add title and labels
ax.set_title('Air temperature in Ås: 1925 vs 2025', fontsize=14)
ax.set_xlabel('Date', fontsize=12)
ax.set_ylabel('Air Temperature (°C)', fontsize=12)

# Rotate the x-axis values and limit to 24
plt.xticks(rotation=45)
ax.xaxis.set_major_locator(ticker.MaxNLocator(nbins=24))

# Add legend and grid
ax.legend(title='Year')
ax.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```



Air temperature in 2025 seems, at first glance, higher than in 1925. The winter period shows the largest difference between 2025 and 1925; temperatures during this period in 2025 were much higher. However, in january, air temperature was much lower in 2025, than in 1925.

Generate a summary table as for task 1, where you display data from both the new and the old temperatures

In [40]:
```python
# Add stats from both years to combined datafame and display
summary_combined = pd.concat([df3['value'].describe(), df['value'].describe()], axis=1)
summary_combined.columns = ['1925', '2025']

display(summary_combined.round(2))
```

	1925	2025
count	364.00	728.00
mean	5.31	7.90
std	9.16	7.53
min	-20.70	-16.10
25%	-1.40	2.80
50%	5.30	8.35
75%	12.42	13.80
max	24.30	24.50

Task 4: Heatwaves

Look for heatwaves in Ås in 2025. In Norway a heatwave is defined as a consecutive period of five or more day where the max temperature is >= 27 degrees Celcius.

Write a function that count the number of heatwaves in a year.

- The function should write a summary to a yml file, formatted as the example file.
- Write the function so that it will work with any station name/id and year

In [49]:
```python
def get_max_temperature_data(station_id, year, client_id):
    """Get max temperature from a given station for a given year"""

    # Retrieve data
    endpoint = 'https://frost.met.no/observations/v0.jsonld'
    parameters = {
        'sources': station_id,
        'elements': 'max(air_temperature P1D)',
        'referencetime': f'{year}-01-01/{year}-12-31',
    }

    response = requests.get(endpoint, params=parameters, auth=(client_id, ''))

    # Check that retrieval was successful, if yes, process into a dataframe
    if response.status_code == 200:
        data = response.json()['data']
        rows = []
        for entry in data:
            date = entry['referenceTime']
            # Check that the observation exists
            if entry['observations']:
                value = entry['observations'][0]['value']
                rows.append({'date': date, 'max_temp': value})

        df = pd.DataFrame(rows)
        # Convert date to datetime-object and sort dataframe
        df['date'] = pd.to_datetime(df['date'])
        df = df.sort_values('date').reset_index(drop=True)
        return df
    # If unsuccesful, raise error
    else:
        print(f'Error retrieving data: {response.status_code} - {response.text}')
        return None


def detect_heatwaves(station_name, station_id, year, client_id, filename='heatwave_summary.yaml'):
    """
    Counts heatwaves (5+ days of max temperature >= 27°C),
    Writes summary to yml-file.
    """

    df = get_max_temperature_data(station_id, year, client_id)
    if df is None or df.empty:
        print("No data found")
        return None

    # Find days that are above 27 degress
    df['above_27'] = df['max_temp'] >= 27

    # Iterate through rows and find consecutive days
    heatwave_ranges = []
    current_streak = []

    for _, row in df.iterrows():
        if row['above_27']:
            current_streak.append(row)
        else:
            if len(current_streak) >= 5:
                start_date = str(current_streak[0]['date'].date())
                end_date = str(current_streak[-1]['date'].date())
                heatwave_ranges.append(f"{start_date} - {end_date}")
            current_streak = []

    if len(current_streak) >= 5:
        start_date = str(current_streak[0]['date'].date())
        end_date = str(current_streak[-1]['date'].date())
        heatwave_ranges.append(f"{start_date} - {end_date}")

    summary = {
        'dates': heatwave_ranges,
        'station_id': station_id,
        'station_name': station_name,
        'year': year
    }

    # Write result to a .yml-file
    with open(filename, 'w', encoding='utf-8') as f:
        yaml.dump(summary, f, allow_unicode=True, sort_keys=False)

    return summary
```

Run for Ås, 2025:

In [50]:
```python
heatwave_summary = detect_heatwaves('Ås', 'SN17850', 2025, user_id)
```

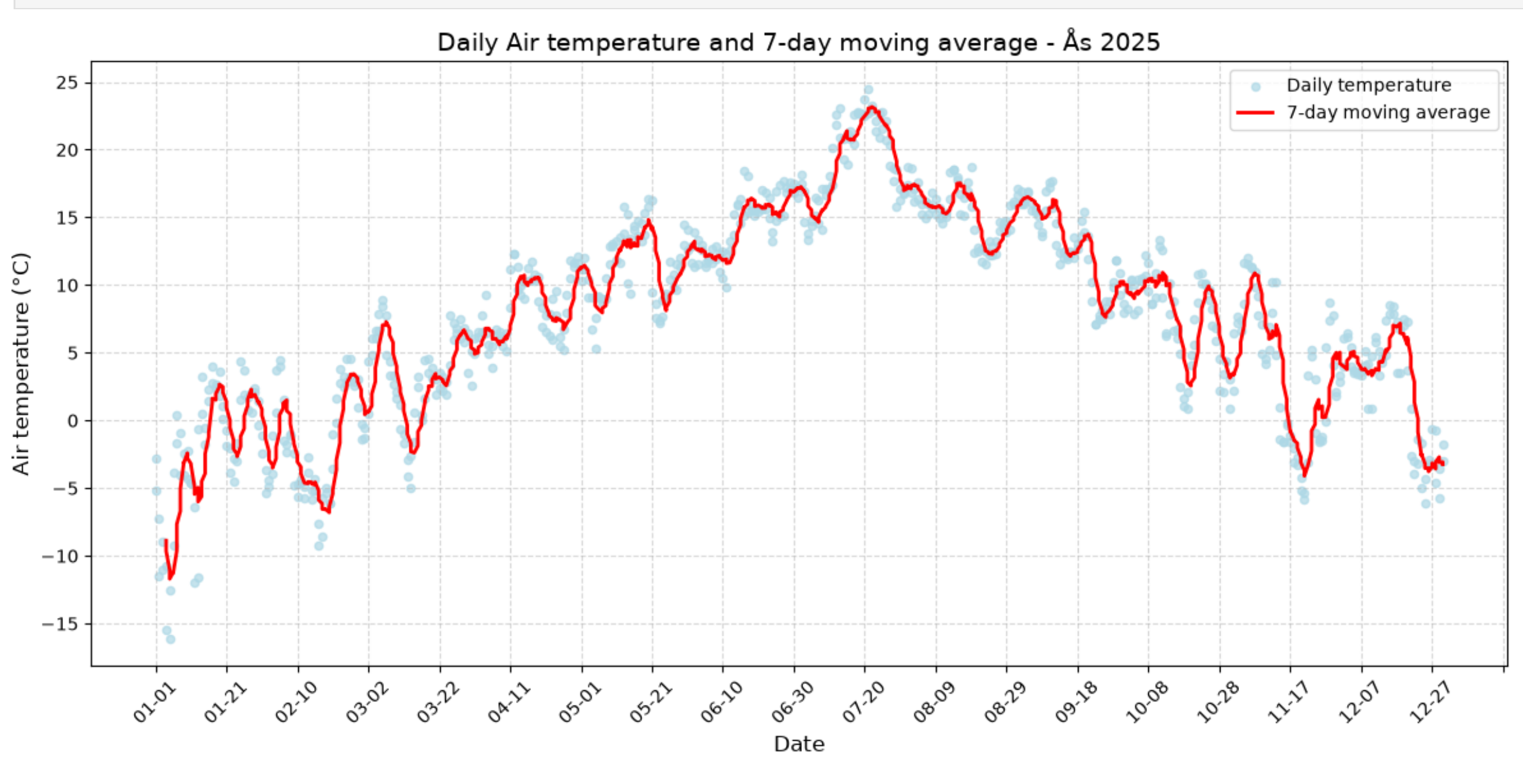
Task 5: Moving average temperature

Write a function that calculates the 7-day moving average for temperature

In [51]:
```python
def calculate_moving_average(df, window_size=7, value_column='value'):
    """
    Calculates moving average for given column in a dataframe
    Standard window-size is 7 days.
    """
    df_ma = df.copy()
    # Use pandas rolling-function to calculate moving average
    df_ma['moving_average'] = df_ma[value_column].rolling(window=window_size).mean()
    return df_ma
```

Use this function on the 2025 mean temperature you collected in task 1 and make a plot showing the moving average as a line and the daily temperature as points

In [55]:
```python
# Calculate moving avergae on 2025 mean temperature
df_ma_airtemp = calculate_moving_average(df, window_size=7, value_column='value')

# Make plot
fig, ax = plt.subplots(figsize=(12, 6))

# Plot daily temperatures using scatter plot
ax.scatter(df_ma_airtemp['month_day'], df_ma_airtemp['value'], label='Daily temperature', color='lightblue', s=20, alpha=0.7)

# Plotmoving average as a line
ax.plot(df_ma_airtemp['month_day'], df_ma_airtemp['moving_average'], label='7-day moving average', color='red', linewidth=2)

# Format plot
ax.set_title('Daily Air temperature and 7-day moving average - Ås 2025', fontsize=14)
ax.set_xlabel('Date', fontsize=12)
ax.set_ylabel('Air temperature (°C)', fontsize=12)

# Rotate the x-axis values and limit to 24
ax.xaxis.set_major_locator(ticker.MaxNLocator(nbins=24))
plt.xticks(rotation=45)

# Add legend and grid
ax.legend()
ax.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```



Comparing the daily average with the daily temperatures, shows that daily temperatures were mostly clustered around the average at the time. In the beginning of the year, there were very low temperatures recorded (approximately -16) for two days.

References:

- https://frost.met.no/python_example.html
- https://stackoverflow.com/questions/37930693/how-can-i-make-a-barplot-and-a-lineplot-in-the-same-plot-with-different-y-axes

Link to GitHub-repo:

https://github.com/theaaga/inf201_deliverables